

Documentación JAVA

Grupo 3

Integrantes:

Enzo Brun
Thiago Cedrés
Franco Da Costa
Camilo Molina
Henry Ford

CI: 5.409.980-3
CI: 5.620.041-4
CI: 5.466.452-7
CI: 5.344.879-8
CI: 2.867.436-0

Introducción.

Este sistema, implementa la lógica de negocio de una pasarela de pagos, que funciona como intermediario entre comercios, medios de pago y entidades financieras como bancos. Su objetivo principal es gestionar de manera segura y modular todo el proceso asociado a pagos electrónicos realizados en comercios adheridos y persistir la información obtenida de dichos procesos en una base de datos local.

El sistema está estructurado en múltiples módulos semi-independientes (pues siguen siendo parte del mismo programa), cada uno con responsabilidades bien definidas, lo que favorece una arquitectura desacoplada y preparada para evolucionar hacia microservicios. Entre las funcionalidades principales se incluyen:

- Registro y administración de comercios y puntos de venta (POS).
- Procesamiento de pagos y generación de reportes de ventas.
- Gestión de transferencias entre medios de pago y cuentas bancarias.
- Notificación de eventos clave para fines de monitoreo.
- Recepción y manejo de reclamos por parte de los comercios.
- Visualización mediante gráficas para monitoreo de los eventos.

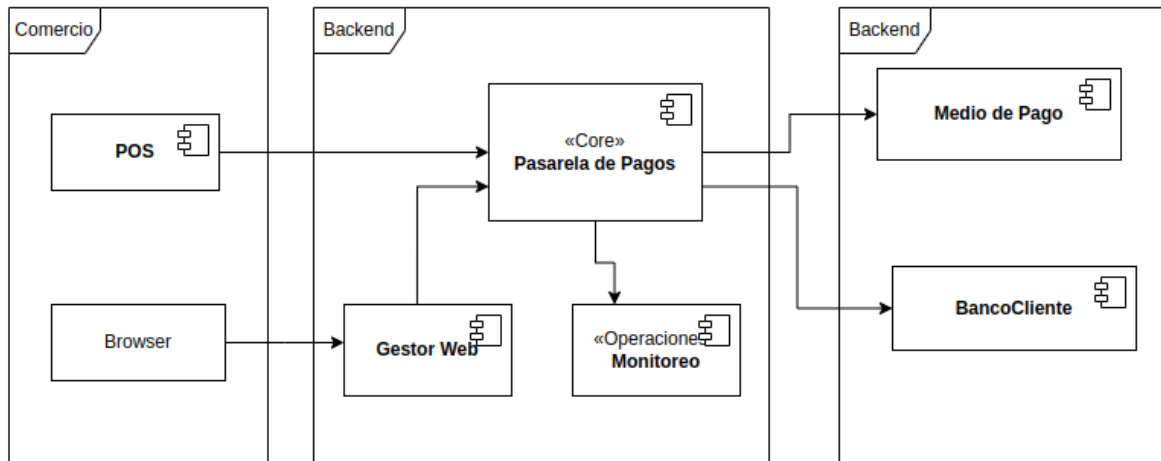
La implementación se hizo siguiendo buenas prácticas de ingeniería de software pautadas a lo largo del taller y requeridas por las letras de las iteraciones durante el desarrollo del programa, como la inyección de dependencias, el uso de eventos para comunicación desacoplada, y una estructura de paquetes clara que separa la lógica de negocio, la interfaz, la infraestructura y la persistencia.

Arquitectura.

Para el funcionamiento correcto de la pasarela de pago se cuenta con varios servicios con los cuales interactúa mediante sus arquitecturas REST o SOAP (en el caso del banco).

Resumen sobre la comunicación con los servicios:

- Pasarela se comunica con Medio de Pago para el chequeo de validez de tarjetas, acto seguido se marca la compra como autorizada, se persisten los datos y se le envían los datos requeridos por el Banco para el depósito de dicha transacción.
- Cada vez que se notifica un evento relevante para el monitoreo como la creación de un reclamo, pago exitoso o error en el proceso de pago, la pasarela se comunicará con los contadores definidos para las gráficas de grafana y se incrementará el contador correspondiente.
- En la creación de un reclamo, la pasarela debe enviar dichos reclamos para ser catalogados por el servicio de etiquetamiento.



Diseño.

La pasarela de pagos posee 4 módulos, comercio, compra, transacción y monitoreo. Cada módulo tiene la siguiente estructura:

src

```

modulo1
  dominio
    repositorio
  aplicación
  interface
  infraestructura
    configuración
    persistencia

modulo2
  ...
  
```

dominio: clases que contienen la lógica del negocio

repositorio: implementación de las interfaces del repositorio

aplicación: interfaces con los servicios que ofrece el módulo y su correspondiente implementación

interface:

interface remotas para acceder a los servicios (no se implementan en esta etapa)

interface para interactuar con otros módulos de forma local

observadores de eventos

infraestructura: código transversal

persistencia: interfaces para la persistencia de datos

Cada módulo posee varias operaciones principales que se pueden llamar desde consola o postman para invocar sus funcionalidades (ya sea por el usuario o la app misma mediante las distintas operaciones que contiene para cumplir con sus funcionalidades)

Módulo de Comercio:

- `altaComercio(datosComercio)` : Permite registrar un comercio en el sistema.
- `modificarDatosComercio(datosComercio)`: Permite modificar información relacionada al comercio.
- `altaPos(comercio, pos)`: Registra un POS en el sistema.
- `cambiarEstadoPos(comercio, pos, estado)`: Permite habilitar/deshabilitar un POS
- `cambioContraseña(nuevaPass)`: Establece, cambia contraseña del comercio. La contraseña se utilizará para poder invocar a la API remota que ofrece información de ventas
- `realizarReclamo(textoReclamo)`: Recibe un reclamo del comercio, el cual deberá ser atendido por el departamento de Soporte.

Módulo Compra :

- `procesarPago(datosCompra)`: procesa el pago de una compra realizada en el Comercio
- `resumenVentasDiarias(comercio)`: retorna información de las ventas realizadas en el día
- `resumenVentasPorPeriodo(comercio)`: retorna información de las ventas realizadas en un período de tiempo
- `montoActualVendido(comercio)`: retorna el acumulado actual de las ventas del día del comercio.

Módulo de Transferencia:

- `recibirNotificacionTransferenciaDesdeMedioPago(datosTransferencia)`: Se invoca desde el sistema de Medios de Pago, informando que se realizó un depósito relacionado a una compra realizada en un comercio. Cuando se recibe la notificación es necesario realizar un depósito en la cuenta del Banco del Cliente, descontado del importe la comisión del Sistema.
- `consultarDepositos(comercio, rangoFechas)`: Devuelve los depósitos realizados para un comercio en un rango de fechas.

Módulo de Monitoreo:

- `notificarPago()`: Se invoca cada vez que se realiza un pago.
- `notificarPagoOk()`: Se invoca cada vez que se realiza un pago exitoso.
- `notificarPagoError()`: Se invoca cada vez que se rechaza un pago.
- `notificarTransferencia()`: Se invoca cada vez que se realiza una transferencia desde el MedioDePago.
- `notificarReclamoComercio()`: Se invoca cuando un comercio realizar un reclamo

Los eventos del módulo de monitoreo son llamados por observadores que los disparan cuando los eventos respectivos son triggereados para subsecuentemente notificar el cambio a las gráficas expuestas en el dashboard de grafana.

Para observabilidad se utiliza grafana como mencionado anteriormente, en conjunto con influxdb el cual se encuentra incluido en la imagen de docker utilizada. Para lograr la comunicación entre la app y las gráficas se aumenta el contador por medio de micrometer, para dicho fin cada módulo lanza eventos mediante un registrador de eventos para que el

observador respectivo del módulo en cuestión dentro del módulo de monitoreo aumente el contador adecuado.

En el caso especial de los reclamos se deben organizar y etiquetar mediante la api de etiquetamiento con la cual nos comunicaremos desde pasarela mediante una queue de messaging.

Es importante tener en cuenta que para comunicarse con la pasarela las solicitudes curl o de postman deben hacerse uso de basic auth, una medida de seguridad básica utilizada en el proyecto.

Módulo de Comercio

Alta comercio: curl -X POST http://localhost:8080/ProyectoJavaee/remota/comercio -H "Content-Type: application/json" -d '{"id": "123", "nombre": "MiTienda", "direccion": "C/Falsa", "telefono": "0999", "email": "a@b.com", "contraseniaHash": "pass", "listaPOS": []}'

Modificar Comercio: curl -X PUT http://localhost:8080/ProyectoJavaee/remota/comercio -H "Content-Type: application/json" -d '{"id": "123", "nombre": "Otro", "direccion": "Av", "telefono": "0999", "email": "x@y.com", "contraseniaHash": "pass", "listaPOS": []}'

Alta Pos: curl -X POST http://localhost:8080/ProyectoJavaee/remota/comercio/123/pos

Cambiar Estado: curl -X PUT "http://localhost:8080/ProyectoJavaee/remota/comercio/123/pos/1/estado?habilitado=true"

Cambiar contraseña: curl -X PUT "http://localhost:8080/ProyectoJavaee/remota/comercio/123/contrasenia?nuevaContrasenia=nueva"

Realizar reclamo: curl -X POST "http://localhost:8080/ProyectoJavaee/remota/comercio/123/reclamo?texto=Problema"

Listar POS: curl -X GET http://localhost:8080/ProyectoJavaee/remota/comercio/123/pos

Módulo de Transferencia

Recibir notificación de transferencia desde medio de pago: curl -X POST http://localhost:8080/ProyectoJavaee/remota/transferencia/notificacion \ -H "Content-Type: application/json" \ -d '{"idCompra": 1, "rutComercio": 123}'

Mostrar depósitos filtrados por periodo: curl -X GET http://localhost:8080/ProyectoJavaee/remota/transferencia/listar?rutComercio=123&fechaini=01-05-2025&fechaend=20-05-2025" \ -H "Content-Type: application/json" \

Módulo Compra

Resumen de ventas diarias: curl -X GET "http://localhost:8080/ProyectoJavaee/api/compras/resumen-diario?idcomercio=123" \ -H "Accept: application/json"

Resumen de ventas por periodo: curl -X GET "http://localhost:8080/ProyectoJavaee/api/compras/resumen-Ventas-Periodo?idcomercio=123&fechaini=01-05-2025&fechaend=20-05-2025" \

```
-H "Accept: application/json"
```

Monto actual vendido en el día: curl -X GET

```
"http://localhost:8080/ProyectoJavaee/api/compras/monto-actual-vendido?idcomercio=123" \
```

```
-H "Accept: application/json"
```

Procesar pago: curl -X POST

```
http://localhost:8080/ProyectoJavaee/api/compras/procesar-pago \
```

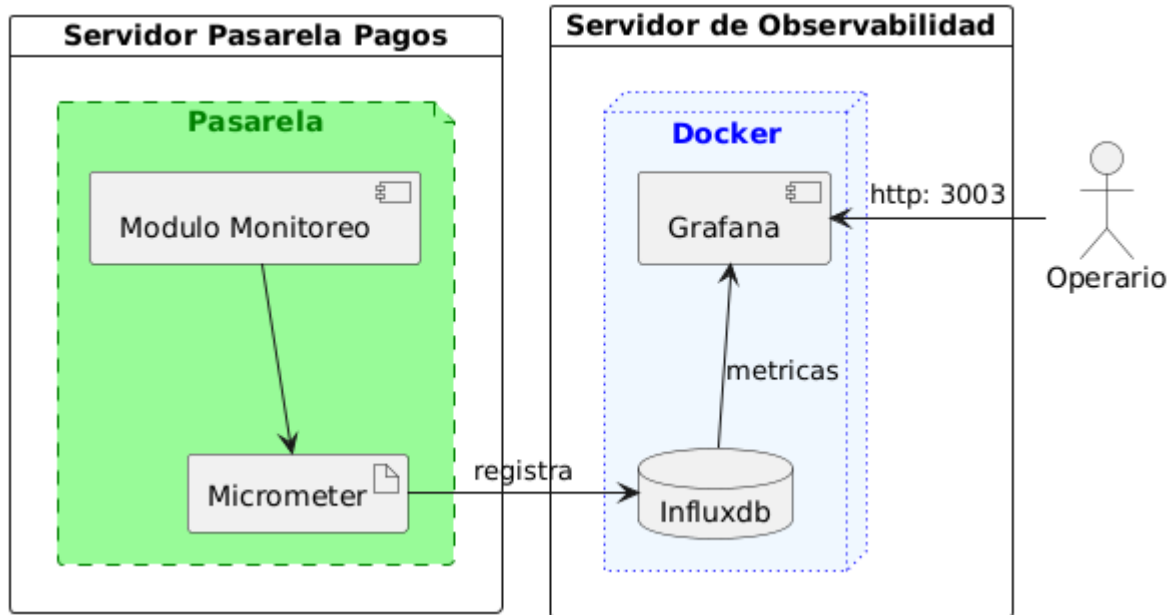
```
-H "Content-Type: application/json" \
```

```
-d '{
  "importe": 1000.0,
  "fecha": null,
  "descripcion": "Pago de producto",
  "tarjeta": {
    "nro": "12345",
    "marca": "VISA",
    "fechaVto": { "dia": 30, "mes": 12, "anio": 2025 }
  },
  "idPos": 1,
  "rutComercio": 123
}'
```

RateLimiter.

La lógica de rate limiter que se implementó regula la cantidad de compras que se pueden procesar por segundo, protegiendo así al sistema de sobrecargas o abusos. Se utiliza un mecanismo de token bucket que permite hasta 10 transacciones acumuladas y recarga 5 por segundo, lo que garantiza que, ante ráfagas de solicitudes, se mantenga un ritmo seguro y estable de procesamiento

Observabilidad.



Para observabilidad se utiliza grafana como mencionado anteriormente, en conjunto con influxdb el cual se encuentra incluido en la imagen de docker utilizada. Para lograr la comunicación entre la app y las gráficas se aumenta el contador por medio de micrometer, para dicho fin cada módulo lanza everton mediante un registrador de eventos para que el observador respectivo del módulo en cuestión dentro del módulo de monitoreo aumente el contador adecuado.





Como se puede ver en las imágenes anexadas anteriormente, el proyecto cuenta con 4 gráficas que muestran cantidad de pagos (ya sea realizados con éxito o con error diferenciados por color), cantidad de reportes, cantidad de transferencias y cantidad de reclamos (diferenciados según si son negativos o no por color).

Link del proyecto: <https://github.com/Cedrego/ProyectoJavaee.git>